

MovieLens Ratings Prediction with NMF

Part 2

Let's load the movie ratings data (**MovieLens 1M**) and use `sklearn.decomposition` module's implementation of non-negative matrix factorization technique (NMF) to predict the missing ratings from the test data. Let's import the required libraries and read the data.

```
In [205]: import pandas as pd
import numpy as np
from sklearn.metrics import mean_squared_error as mse

MV_users = pd.read_csv('data/users.csv')
MV_movies = pd.read_csv('data/movies.csv')
train = pd.read_csv('data/train.csv')
test = pd.read_csv('data/test.csv')

print(MV_users.shape, MV_movies.shape, train.shape, test.shape)
train.head()
#test.head()
```

```
(6040, 5) (3883, 21) (700146, 3) (300063, 3)
```

Out[205]:

	uID	mID	rating
0	744	1210	5
1	3040	1584	4
2	1451	1293	5
3	5455	3176	2
4	2507	3074	5

```
In [213]: #sorted(train.uID.unique())
train[(train.uID == 5) & (train.mID == 6)]
```

Out[213]:

	uID	mID	rating
334454	5	6	2

If we pivot the dataset, as can be seen, the data contains a huge number of missing values (`NaN`) values, the rating matrix being very very **sparse**.

```
In [254]: train_ratings = train.pivot(index='uID', columns='mID', values='rating')
print(train_ratings.shape)
train_ratings.head()
```

(6040, 3664)

Out[254]:

	mID	1	2	3	4	5	6	7	8	9	10	...	3943	3944	3945	3946	3
uID																	
1	5.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	↑
2	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	↑
3	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	↑
4	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	↑
5	NaN	NaN	NaN	NaN	NaN	2.0	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	↑

5 rows × 3664 columns



Limitation(s) of sklearn's non-negative matrix factorization library

The NMF implementation **does not** handle **missing values**. Hence we need to fill (impute) the missing values (with non-negative values) before we can use the NMF implementation to generate the ratings for the movies that are not rated by the users, using NMF .

- To start with, let's fill the missing ratings in the `train` dataset by **zeros** (for example with the function `impute_missing()`).
- Then train an NMF model with `k` components (`k = 10` , for example) on the `train` dataset.
- Use the NMF model fitted on `train` dataset to predict the ratings for the `test` dataset (with `get_NMF_pred()`).
- Compare the prediction error with RMSE (with the function `get_pred_RMSE()`).

```
In [248]: missing_locs = np.isnan(train_ratings)
train_ratings[missing_locs] = 0
train_ratings.head()
```

Out[248]:

	mID	1	2	3	4	5	6	7	8	9	10	...	3943	3944	3945	3946	3947	3948	:
uID																			
1	5.0	3.0	3.0	3.0	3.0	3.0	3.0	3.0	3.0	3.0	3.0	...	3.0	3.0	3.0	3.0	3.0	3.0	
2	3.0	3.0	3.0	3.0	3.0	3.0	3.0	3.0	3.0	3.0	3.0	...	3.0	3.0	3.0	3.0	3.0	3.0	
3	3.0	3.0	3.0	3.0	3.0	3.0	3.0	3.0	3.0	3.0	3.0	...	3.0	3.0	3.0	3.0	3.0	3.0	
4	3.0	3.0	3.0	3.0	3.0	3.0	3.0	3.0	3.0	3.0	3.0	...	3.0	3.0	3.0	3.0	3.0	3.0	
5	3.0	3.0	3.0	3.0	3.0	2.0	3.0	3.0	3.0	3.0	3.0	...	3.0	3.0	3.0	3.0	3.0	3.0	

5 rows × 3664 columns



```

In [273]: def impute_missing(ratings, val):
            ratings[np.isnan(ratings)] = val
            return ratings

def get_NMF_pred(train, test, impute_val = 0, impute_missing = impute_missing, k = 10):
    # impute
    train_ratings = train.pivot(index='uID', columns='mID', values='rating')
    train_ratings = impute_missing(train_ratings, impute_val) # replace NaN values
    # train NMF
    nmf = decomposition.NMF(
        n_components=k,
        random_state=0,
        init = "nndsvda",
        beta_loss="frobenius",
        #alpha_W=0.001,
        #alpha_H=0.001,
    )
    W1 = nmf.fit_transform(train_ratings)
    H1 = nmf.components_
    print(f'NMF reconstruction error: {nmf.reconstruction_err_}')
    # predict with NMF
    pred = W1 @ H1
    pred_df = pd.DataFrame(data = pred,
                           index = train_ratings.index.values,
                           columns = train_ratings.columns.values)
    pred_df['uID'] = pred_df.index.values
    #print(pred_df.head())
    pred_df = pd.melt(pred_df, id_vars=['uID'], var_name='mID', value_name='pred_rating')
    out_df = pred_df.merge(test, on=['uID', 'mID'])
    out_df.head()
    return out_df[['uID', 'mID', 'rating', 'pred_rating']]

def get_pred_RMSE(pred_df):
    return np.sqrt(mse(pred_df['rating'].values, pred_df['pred_rating'].values))

```

Impute the missing values in the training dataset with zeros, train NMF and predict

```
In [275]: pred_df = get_NMF_pred(train, test)
pred_df.head()
```

NMF reconstruction error: 2692.0484632343446

Out[275]:

	uID	mID	rating	pred_rating
0	6	1	4	0.717425
1	8	1	4	0.704151
2	21	1	3	0.253164
3	23	1	4	1.319983
4	26	1	3	1.377998

RMSE with the NMF model

```
In [276]: print(f'RMSE: {get_pred_RMSE(pred_df)}')
```

RMSE: 2.911772946856598

RMSE with the Baseline model predict_everything_to_3

```
In [277]: pred_df['pred_rating'] = 3
print(f'RMSE: {get_pred_RMSE(pred_df)}')
```

RMSE: 1.2585673019351262

As can be seen from above, the prediction with NMF is poorer than the baseline model, comparing in terms of RMSE .

NMF results in poor performance since the matrix is very very **sparse** and all the missing ratings are kept as **zeros** in the matrix being factorized ($X = WH$). With the Frobenius norm loss function, the predicted ratings are pushed towards zero, which is incorrect.

Ways to improve the prediction with the NMF model

The issue can be fixed if the missing components from the rating matrix can be masked out when the loss function is computed, but currently, the `sklearn` 's NMF implementation does not allow to change the loss function.

One way to fix this issue is to impute the missing values differently, for example, using the next two approaches, which improves the prediction RMSE a lot.

1. RMSE with the NMF model by imputing the missing values with rating value 3

```
In [278]: pred_df = get_NMF_pred(train, test, impute_val = 3)
          print(f'RMSE: {get_pred_RMSE(pred_df)}')
```

NMF reconstruction error: 995.1352921379064

RMSE: 1.1500766186214655

2. RMSE with the NMF model by imputing the missing ratings for an item by average user ratings for the item

As described here (<https://stackoverflow.com/questions/39367597/how-to-deal-with-missing-values-in-python-scikit-nmf/77255743#77255743> (<https://stackoverflow.com/questions/39367597/how-to-deal-with-missing-values-in-python-scikit-nmf/77255743#77255743>)), we can impute the missing ratings for an item by average user ratings for the item.

```
In [279]: def impute_missing_avg_item_rating(ratings, val=None):
          missing_locs = np.isnan(ratings)
          mean = ratings.apply(np.nanmean, axis=0)
          ratings.fillna(mean, inplace=True)
          return ratings

          pred_df = get_NMF_pred(train, test, impute_missing = impute_missing_avg_item_rating, impute_val = None)
          print(f'RMSE: {get_pred_RMSE(pred_df)}')
```

NMF reconstruction error: 807.7526430141833

RMSE: 0.9651849775012515